exp-orchestrator: project summary

What I built, why I built it that way, and where it is going. Prepared 15 May 2026.

Repo	sean-lai-sh/exp-orchestrator
Stack	Next.js + React Flow frontend, FastAPI backend, Docker executors, Convex (auth and persistence)
Author	Sean Lai (37 commits, project owner and primary contributor through April 2026)
Span	Initial commit 28 May 2025 to current work May 2026, roughly one year of incremental shipping.

1. What the product is

exp-orchestrator is a visual orchestration tool for experimental data workflows. A researcher opens a canvas, drops nodes for senders, plugins, and receivers, wires them with typed edges, and clicks Deploy. The backend converts that graph into a runnable plan: it topologically sorts the DAG, mints stream credentials per edge, validates that every plugin image is on an allowlist, and hands execution to a pluggable executor (local Docker today, ECS and Kubernetes stubs in place). The goal is to give scientists the ergonomics of Node-RED with the rigor of a real orchestrator, without forcing them to learn Airflow or Kubernetes.

2. What I built

Foundations (May to July 2025)

- Bootstrapped the Next.js canvas on React Flow, with custom editable nodes, a left control panel, and a right property panel.
- Built the type-compatibility engine for edges so connections that would silently break at runtime are flagged at authoring time. Rules live in `lib/CompatRules` and are merged with user rules; an unsafe-rule guard prevents users from marking risky coercions as ok by accident.
- Added secure-token handling for node credentials with reveal, copy, and toast feedback, so sensitive material is never exposed by default in the UI.

Deploy pipeline and DAG engine (July 2025 to Feb 2026)

- Stood up the FastAPI backend with a single POST /deploy endpoint that accepts the same DeployWorkflow contract the canvas emits.
- Wrote the DAG layer from scratch: Kahn-style topological sort with cycle detection in `backend/dag.py`, plus a deploy planner that builds the adjacency list, queues only plugin nodes, and emits a per-node env plan including injected credentials.
- Added Docker-based execution: built test images, wrote a container-level integration script, and confirmed end-to-end that plugin nodes receive their generated env vars when launched.

Hardening, CI, and product surface (Feb to April 2026)

- Authored the multi-job GitHub Actions pipeline (backend pytest, image build with load:true so the image lands in the local daemon, linting) so every PR runs the full backend suite.
- Shipped a reference plugin and an upload-validation flow so outside contributors can submit plugins against a known contract.

- Built the image-allowlist gate (`backend/allowlist.py`) and wired it into the deploy flow so unknown container images are rejected before they ever start.
- Wrote a standalone DAG analyzer and retired stale editor variants that had accumulated, cutting roughly 3000 lines of dead code in one pass (PR #38).
- Added /health and /health/ready endpoints so the orchestrator is deployable behind a real load balancer rather than just on a laptop (PR #65).

Architecture direction (April 2026 onward)

- Drafted two design docs in `docs/plans/`: a post-Corelink architecture that replaces the demo transport with NATS plus JetStream and S3-pointer payloads, and a broader R and D lab framing that recasts the product around lab workers, instruments, and sessions rather than generic MLOps.

3. Engineering decisions I made

Single shared deploy contract

The frontend canvas and the backend planner speak one Pydantic schema, `DeployWorkflow`, documented in `docs/deploy-contract.md`. Picking a shared shape early meant I never had to translate React Flow's node graph into a second representation; the canvas serializes directly to what the backend validates. The cost is that frontend and backend evolve in lockstep, but for a two-side project run by a small team that is the right trade.

Type-checked edges at authoring time

Plugins produce and consume typed streams (json, bytes, parquet, tensor, and so on). I built the compatibility engine to evaluate edges as the user wires them, returning ok, warn, or error, and surfacing the result as a toast. The reasoning was simple: an experimentalist who only finds out at deploy time that their tensor cannot flow into a json sink will blame the product, not their workflow. The engine ships with default rules, allows user rules to layer on top, and refuses to let users mark unsafe coercions as ok unless the target is the bytes catch-all.

DAG as the canonical execution primitive

I chose a directed acyclic graph as the only legal workflow shape. Cycle detection runs on every deploy and rejects the workflow with a clear error. This rules out a class of bugs (live-locks, fan-back loops) at the planner level rather than at runtime, and it lets the planner emit a topological order that downstream executors can trust. Streaming workflows that look cyclic are modeled as separate forward edges through a broker, not as graph cycles.

Image allowlist as a deploy-time gate

Plugins are Docker images, which means a careless workflow could pull and run arbitrary code on the orchestrator host. I added an allowlist check that runs before any executor is invoked: every runtime image is matched against `config/allowed_images.json`, with support for exact references and prefix patterns. A deploy with an unapproved image is rejected with a structured reason rather than silently degrading. This is intentionally a hard wall, not a warning; for an experimental platform the blast radius of a bad image is too large to leave permissive.

Executor abstraction over concrete backends

The deploy planner does not know how containers actually start. Every backend implements the same `Executor` ABC: start, stop, status, logs, health_check. LocalDockerExecutor and NoopExecutor exist today; ECS is stubbed; Kubernetes and a worker-agent model are scoped for the next phase. The cost of the indirection is small (one file per backend) and the payoff is that adding a new scheduler does not require touching the planner, the canvas, or the deploy contract.

Convex for auth and project persistence, no SQL

Rather than stand up Postgres plus a separate auth service, I put auth (better-auth) and project storage on Convex. Workflows are stored as serialized React Flow state per project, autosaved on a two-second debounce. The decision favored shipping speed over long-term flexibility; the document model fits the React Flow shape almost perfectly, and migrating to a relational store later is cheap because the contract is just JSON.

CI that runs the real backend, not just lint

Backend tests use fresh DeployWorkflow fixtures per case; the Docker env-injection test mocks the container start but still asserts that the temporary compose file is cleaned up. CI builds the plugin image with `load: true` so the image actually lands in the daemon and the integration job can pull from local. I treated CI as part of the product because without it the deploy planner is impossible to refactor with confidence.

Health endpoints split into liveness and readiness

/health answers whether the process is up; /health/ready answers whether downstream dependencies are reachable. Splitting them matters once the orchestrator runs behind a load balancer, where liveness drives restarts and readiness drives traffic routing. Conflating them is a common subtle bug I wanted to rule out before the product moves off a laptop.

4. Where the project is going

The two design docs in `docs/plans/` lay out the next two big bets. The post-Corelink doc commits to swapping the demo transport for NATS plus JetStream as the broker and S3 or MinIO as the payload store, with the broker carrying small JSON pointers and the object store carrying tensors. This fixes the three sharpest pains of the current stack: state accumulation in the demo transport, single-process data plane, and CPU cost of serializing tensors through WebSocket frames. The R and D lab doc reframes the product around lab workers, instruments, and sessions, and treats Kubernetes as an eventually-available option rather than the v1 scheduler.

Near-term roadmap

- Phase A: stand up the NATS path alongside the demo transport, behind a `broker=nats` query param so we can switch per-deploy without breaking the demo.
- Phase B: flip the default to NATS and document the cutover.
- Phase C: rip the demo transport per the deletable-files checklist in the design doc. The original demo spec scoped transport code into named files so the rip-out is an enumerated delete, not a refactor.
- Phase D: flesh out the ECS executor stub, then add a Kubernetes executor that turns canvas requirements blocks into PodSpecs with GPU resource requests.

Open questions I have not resolved yet

- Authn at the broker layer: per-deploy tokens are simplest, JWTs or NKeys are forward-compatible. I have not chosen.
- Schema validation at plugin boundaries: opt-in JSON Schema at runtime, or trust the canvas-time type check only.
- Backpressure surfacing on the canvas: showing a backed-up edge indicator requires the orchestrator to poll consumer lag. Worth the visibility, but adds a polling loop.

5. Closing

The shape of the product is now clear: a typed canvas, a deploy planner that mints credentials and rejects unsafe images, and a pluggable executor layer that can grow from local Docker to real lab compute. The next phase replaces the demo transport with a broker built for streaming, moves heavy payloads to object storage, and pushes the product toward its real home in R and D labs. The foundations I committed to (single contract, DAG-only, allowlist-by-default, executor as an abstraction) hold up under that direction, which is the best signal I have that the early engineering calls were the right ones.